
Advanced Computer Architectures

June 10, 2025

Prof. Christian Pilato

Name
Last Name
ID Number (codice persona)

Problem 1 (15%)	
Problem 2 (20%)	
Problem 3 (15%)	
Question 1 (15%)	
Question 2 (20%)	
Question 3 (15%)	
Total (100%)	

NOTE: students have 90' to complete the exam.

To pass the midterm, a minimum score of 20% is requested by each “section” (section 1: problems, section 2: questions).

The overall score to access the oral has to be greater or equal to 50%

Problem 1

In this problem, you will port code to a simple **5-operations VLIW machine**, and schedule it to improve performance.

Details about the 5-operations VLIW machine with 5 fully pipelined multi-cycle functional units:

- 1 Integer ALU with 1 cycle latency
- 2 Memory Unit with 3 cycles latency
- 1 Floating Point Unit (FPU) SUM with 3 cycles latency
- 1 FPU MUL with 5 cycles latency

Assembly:

```
FOR: LD $F2, A($R1)
I1: LD $F4, B($R1)
I2: LD $F6, C($R1)
I3: FADD $F4, $F2, $F4
I4: FADD $F6, $F4, $F6
I5: SD $F6, A($R1)
I6: ADDUI $R1, $R1, 8
I7: BNE $R1, $R2, FOR
```

P1.A Considering **one iteration** of the loop, **schedule** the assembly code for the 5-operations VLIW machine in the following table by using the as soon as possible (ASAP) **scheduling**, but neither using software pipelining nor loop unrolling **nor modifying loop indexes**. Please do not need to write in NOPs (can leave blank).

P1.B Compute FLOPs/CC.

P1.C Consider unrolling by a factor of 2 – i.e., double each instruction - and the FPU mult could execute also SUM with 3 cc latency, how would be affected the performance?

Time	ALU1cc	MU 3cc	MU 3cc	FPU + 3 cc	FPU * 5cc
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					
16					
17					
18					
19					
20					
21					

Answer P1.B

Answer P1.C

Problem 2

Assume that the following code has been executed on a CPU with Tomasulo.

	Instruction	ISSUE	START EXE	WB	RS	FU
I1	ld \$f1,FORCE(\$fp)	1	2	4		
I2	ld \$f0,DY(\$fp)	2	3	5		
I3	mul.d \$f0,\$f1,\$f0	3	6	16		
I4	ld \$f1,FY(\$fp)	4	5	8		
I5	add.d \$f0,\$f1,\$f0	5	17	20		
I6	sd \$f0,FY(\$fp)	6	21	23		
I7	ld \$f1,FORCE(\$fp)	7	8	10		
I8	ld \$f0,DZ(\$fp)	8	9	11		
I9	mul.d \$f0,\$f1,\$f0	9	12	22		
I10	ld \$f1,FZ(\$fp)	10	11	13		
I11	add.d \$f0,\$f1,\$f0	11	23	26		
I12	sd \$f0,FZ(\$fp)	12	27	29		
I13	lw \$2,28(\$fp)	13	14	16		
I14	addi \$2,\$2,1	14	17	18		
I15	sw \$2,28(\$fp)	15	19	21		

A. List all the possible true data dependencies – and only them - in the code.

B. Is there a Tomasulo “configuration” (i.e., FUs and RSs) that can respect the shown execution? Yes or No? Please explain why and use the RS and FU column (if needed).

Assume that all the memory operations are certainly without any ambiguity.

C. If the previous table was not correct – i.e., previous answer is no -, please, write the right architectural configuration and specify functional unit number, kind and latency for each unit and reservation stations. Not writing any of these information will be considered as an error.

D. Calculate the CPI (either point B. or C.)

Answer 2.A

Answer 2.B

Answer 2.C

	Instruction	ISSUE	START EXE	WB	RS	FU
I1	ld \$f1,FORCE(\$fp)					
I2	ld \$f0,DY(\$fp)					
I3	mul.d \$f0,\$f1,\$f0					
I4	ld \$f1,FY(\$fp)					
I5	add.d \$f0,\$f1,\$f0					
I6	sd \$f0,FY(\$fp)					
I7	ld \$f1,FORCE(\$fp)					
I8	ld \$f0,DZ(\$fp)					
I9	mul.d \$f0,\$f1,\$f0					
I10	ld \$f1,FZ(\$fp)					
I11	add.d \$f0,\$f1,\$f0					
I12	sd \$f0,FZ(\$fp)					
I13	lw \$2,28(\$fp)					
I14	addi \$2,\$2,1					
I15	sw \$2,28(\$fp)					

Answer 2.D

Problem 3 (15%)

Consider the following access pattern on a three-processor system with a direct-mapped, write-through cache with one cache block and a three cache block memory. Use MESI, with write-through caches, write-allocate, and write-invalidate of other caches.

Time	After Operation	Px cache block state	Py cache block state	Pz cache block state	Memory at Block 0 up to date?	Memory at Block 1 up to date?	Memory at Block 2 up to date?
1	Px read block 2	Exclusive (0)	Shared (2)	Shared (2)	yes	yes	yes
2	Pz write block 2						
3	Px read block 2						
4	Pz read block 1						
5	Py write block 2						
6	Px read block 2						
7	Py write block 1						
8	Pz read block 0						
9	Px write block 0						
10	Py read block 2						
11	Py read block 2						
12	Pz write block 1						
13	Px read block 1						
14	Pz read block 0						
15	Pz read block 1						
16	Py read block 1						
17	Px write block 1						
18	Px read block 1						
19	Pz write block 1						

